Models in ASP.Net Core WebAPI

Models

- In ASP.NET Core Web API, models serve as the backbone for representing and managing data within an application.
- They define
  - The structure of data
  - Apply validation rules to ensure integrity
  - Establish relationships between entities when working with databases

Models

- Models serve as blueprints that guide the flow of data between the client and server
- Models are essential for
  - Handling client requests
  - Processing business logic
  - Persisting information in a reliable and organized manner.

Key Features of Models

- **Data Representation (POCOs)**
  - Models are typically **POCOs (Plain Old CLR Objects)**, simple classes that do not depend on any base class or framework inheritance.
  - They define properties that reflect the data structure of the application or API.
  - These classes form the foundation for handling real-world entities, such as employees, Products, or Orders.

Key Features of Models

- **Data Validation with Data Annotations**
  - Models can be **annotated with attributes** from the System.ComponentModel.DataAnnotations namespace to apply validation rules. These rules validate incoming data automatically before it reaches business logic.
  - **Common Data Annotations:**
    - [Required] → ensures a property is not null or empty.
    - [StringLength(50)] → restricts text length.
    - [Range(1, 100)] → enforces numeric ranges.
    - [RegularExpression] → checks formats (e.g., email, phone numbers).
  - This reduces bugs and ensures the **integrity of incoming data**.

Key Features of Models

- **Defining Relationships (EF Core Models)**
  - When used with **Entity Framework Core**, models can define relationships such as:
    - **One-to-Many**
    - **One-to-One**
    - **Many-to-Many**
- This is done using **Navigation Properties** and foreign key conventions.
- For example, A Product belongs to one Category, while a Category can have many Products (one-to-many).

Key Features of Models

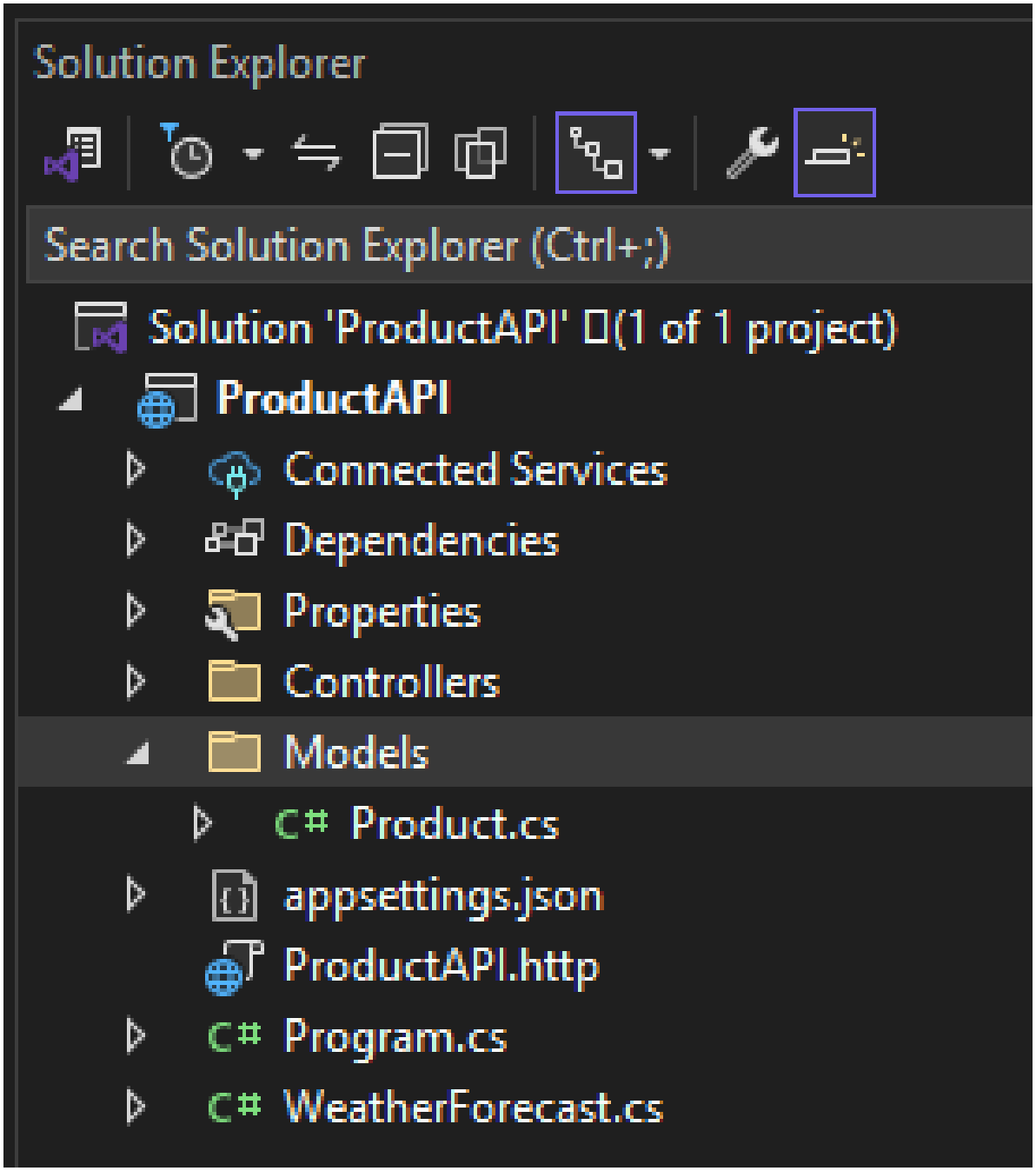
- **DTOs (Data Transfer Objects)**
  - **DTOs** are **not Entity Models** — they are **Lightweight Classes** designed to:
    - Models can also act as **DTOs**, which are specialized objects for shaping the data exposed by the API.
    - Hide sensitive/internal fields (like passwords or internal IDs) and only return what the client actually needs.
    - DTOs improve **security, performance, and clarity** of API responses by sending only required data.

Models: Real World Analogy

- Think of **Models** as the **blueprint of a house**:
  - They define what rooms (properties) exist, how rooms connect (relationships), and the rules for construction (validation, i.e., size, safety rules).
  - Entity Framework is like the **Builder** who uses this blueprint to construct the actual house (database tables).
  - DTOs are like **sales brochures**; you don't show the wiring and plumbing (sensitive data), only what the customer needs to see.

Creating Models

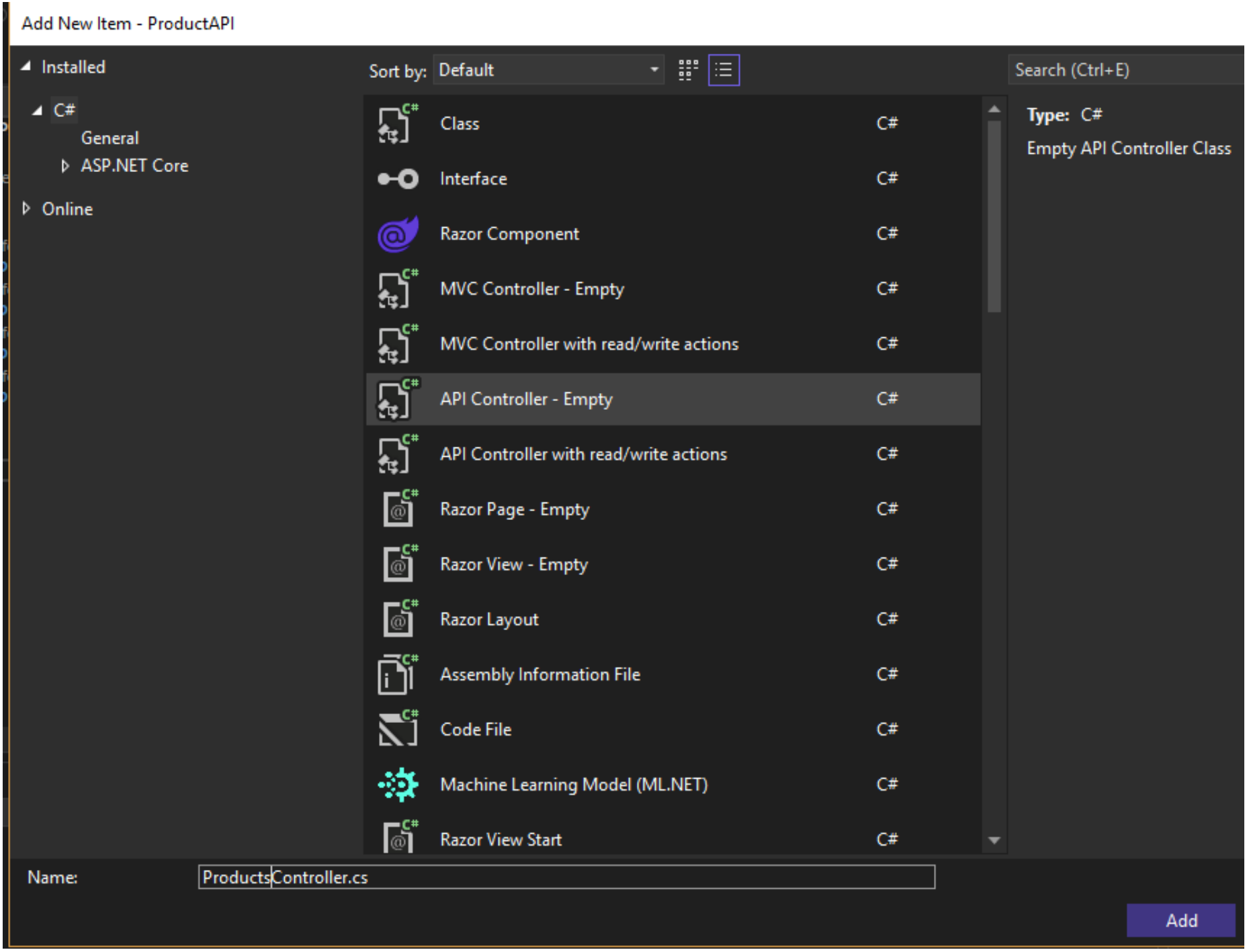
- Models can technically be placed **anywhere**, but for clarity and maintainability, you can create models:
  - In a **Dedicated Models folder** within the project (recommended for clarity).
  - In **any Folder/Subfolder** of the project (common in small projects).
  - In a **Separate Class Library Project,** such as MyApp.Domain or MyApp.Models (preferred for larger applications, as they keep domain models independent of the API).


Solution Explorer
Search Solution Explorer (Ctrl+;)
Solution 'ProductAPI' (1 of 1 project)
ProductAPI
Connected Services
Dependencies
Properties
Controllers
Models
Product.cs
appsettings.json
ProductAPI.http
Program.cs
WeatherForecast.cs

Product.cs

ProductAPI

```
1  namespace ProductAPI.Models
2  {
3      public class Product
4      {
5          public int Id { get; set; }
6          public string Name { get; set; }
7          public decimal Price { get; set; }
8          public string Category { get; set; }
9      }
10 }
```


Add New Item - ProductAPI
Installed
C#
General
ASP.NET Core
Online
Sort by: Default
Search (Ctrl+E)
Class C#
Interface C#
Razor Component C#
MVC Controller - Empty C#
MVC Controller with read/write actions C#
API Controller - Empty C#
API Controller with read/write actions C#
Razor Page - Empty C#
Razor View - Empty C#
Razor Layout C#
Assembly Information File C#
Code File C#
Machine Learning Model (ML.NET) C#
Razor View Start C#
Type: C#
Empty API Controller Class
Name: ProductsController.cs
Add

```
using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Mvc;
using ProductAPI.Models;

namespace ProductAPI.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    0 references
    public class ProductsController : ControllerBase
    {
        private static List<Product> _products = new List<Product>
        {
            new Product { Id = 1, Name = "Laptop", Price = 1000.00m, Category = "Electronics" },
            new Product { Id = 2, Name = "Desktop", Price = 2000.00m, Category = "Electronics" },
            new Product { Id = 3, Name = "Mobile", Price = 3000.00m, Category = "Electronics" },
        };
```

```
// GET: api/products
[HttpGet]
0 references
public ActionResult<IEnumerable<Product>> GetProducts()
{
    return _products;
}

// GET: api/products/2
[HttpGet("{id}")]
1 reference
public ActionResult<Product> GetProduct(int id)
{
    var product = _products.FirstOrDefault(p => p.Id == id);
    if (product == null)
    {
        return NotFound();
    }
    return product;
}
```

GET

- **[HttpGet]:** Indicates that this action handles HTTP GET requests.
- **ActionResult<IEnumerable<Product>>:** The action returns an IEnumerable of Product, wrapped in an ActionResult for HTTP response flexibility.
- **ActionResult<Product>:** The action returns a Product, wrapped in an ActionResult for HTTP response flexibility.
- **[HttpGet("{id}")]:** Specifies that this action responds to a GET request with an id parameter in the route.
- This method searches for a product with the given id. If not found, it returns a 404 Not Found response.

POST

```
// POST: api/products
[HttpPost]
0 references
public ActionResult<Product> PostProduct(Product product)
{
    _products.Add(product);
    return CreatedAtAction(nameof(GetProduct), new { id = product.Id }, product);
}
```

[HttpPost]: Indicates handling of POST requests.

CreatedAtAction: Returns a 201 Created response, and the Location header includes the URI of the new product.

PUT

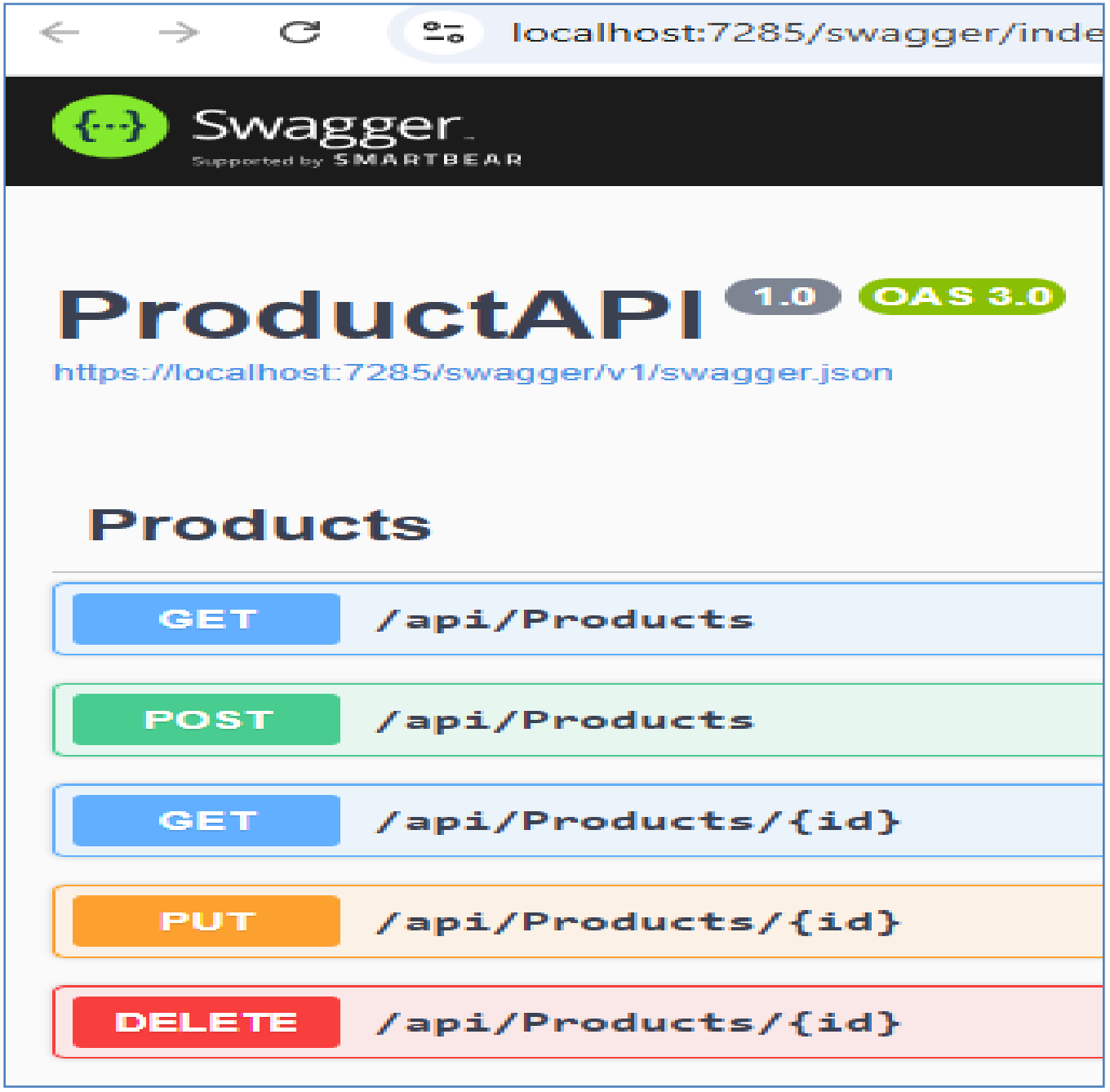
```
// PUT: api/products/5
[HttpPut("{id}")]
0 references
public IActionResult PutProduct(int id, Product product)
{
    if (id != product.Id)
    {
        return BadRequest();
    }
    var existingProduct = _products.FirstOrDefault(p => p.Id == id);
    if (existingProduct == null)
    {
        return NotFound();
    }
    existingProduct.Name = product.Name;
    existingProduct.Price = product.Price;
    existingProduct.Category = product.Category;
    // In a real application, here you would update the product in the database
    return NoContent();
}
```

[HttpPut(“{id}”)]: Specifies that this action responds to a PUT request and expects an id in the route.

DELETE

```
// DELETE: api/products/5
[HttpDelete("{id}")]
0 references
public IActionResult DeleteProduct(int id)
{
    var product = _products.FirstOrDefault(p => p.Id == id);
    if (product == null)
    {
        return NotFound();
    }
    _products.Remove(product);
    // In a real application, here you would delete the product from the database
    return NoContent();
}
```

[HttpDelete(“{id}”)]: Indicates that this action handles DELETE requests with an id parameter.


localhost:7285/swagger/inde
Swagger
Supported by SMARTBEAR
ProductAPI
1.0
OAS 3.0
https://localhost:7285/swagger/v1/swagger.json
Products
GET
/api/Products
POST
/api/Products
GET
/api/Products/{id}
PUT
/api/Products/{id}
DELETE
/api/Products/{id}

Request URL

```
https://localhost:7285/api/Products
```

Server response

Code	Details
200	Response body


```
[
  {
    "id": 1,
    "name": "Laptop",
    "price": 1000,
    "category": "Electronics"
  },
  {
    "id": 2,
    "name": "Desktop",
    "price": 2000,
    "category": "Electronics"
  },
  {
    "id": 3,
    "name": "Mobile",
    "price": 3000,
    "category": "Electronics"
  }
]
```


Download

Request URL

```
https://localhost:7285/api/Products/2
```

Server response

Code	Details
200	Response body


```
{
  "id": 2,
  "name": "Desktop",
  "price": 2000,
  "category": "Electronics"
}
```


Download

POST /api/Products

Parameters

Cancel

Reset

No parameters

Request body

application/json

```
{
  "id": 4,
  "name": "Tablet",
  "price": 6000,
  "category": "Electronics"
}
```

Execute

Request URL

```
https://localhost:7285/api/Products
```

Server response

Code	Details
201 *Undocumented*	Response body


```
{
  "id": 4,
  "name": "Tablet",
  "price": 6000,
  "category": "Electronics"
}
```


Download

Request URL

```
https://localhost:7285/api/Products/4
```

Server response

Code	Details
200	Response body


```
{
  "id": 4,
  "name": "Tablet",
  "price": 6000,
  "category": "Electronics"
}
```


Download

PUT /api/Products/{id}

Parameters

Cancel

Reset

Name	Description
id * required integer($int32) *(path)*	4

Request body

application/json

```
{
  "id": 4,
  "name": "Tablet",
  "price": 20000,
  "category": "Electronics"
}
```

Request URL

```
https://localhost:7285/api/Products/4
```

Server response

Code	Details

204
Undocumented
	Response headers
	`date: Sat,14 Dec 2024 07:23:29 GMT`
	`server: Kestrel`


DELETE
/api/Products/{id}
Parameters
Cancel
Name
Description
id * required
integer($int32)
(path)
4
Execute

Request URL

```
https://localhost:7285/api/Products/4
```

Server response

Code	Details

204
Undocumented
	Response headers
	`date: Sat,14 Dec 2024 07:26:26 GMT`
	`server: Kestrel`

Request URL

```
https://localhost:7285/api/Products/4
```

Server response

Code	Details
404 *Undocumented*	Error: response status is 404 Response body


```
{
  "type": "https://tools.ietf.org/html/rfc9110#section-15.5.5",
  "title": "Not Found",
  "status": 404,
  "traceId": "00-913ff092da7c0ccfee19dcb87450733f-1ce07c132c8
}
```


Download

Usage of Models

- **Binding Data:** Models are used to bind incoming data from HTTP requests to strongly typed objects that API controllers can process.
- **Returning Data:** Models are also used to return data in HTTP responses, providing a structured format that clients can understand.
- **DTOs (Data Transfer Objects):** In the context of Web APIs, models are used as Data Transfer Objects (DTOs). DTOs are objects that carry data between processes, in this case, between the client and the server.

Usage of Models

- By using DTOs, we can customize which data is sent over the network and optimize interaction between the client and server.
- **Validation:**
  - Models often include validation rules that ensure data integrity before it's processed by the application or saved to a database.
  - ASP.NET Core uses data annotations and other validation frameworks to automate this process.
  - If the model state is invalid, the API returns a response with the validation errors.